

Informe Técnico - Problema de la Mochila 0/1 con Programación Dinámica

Programación Avanzada • ICC708 • Semana 9

1. Introducción

En muchos problemas reales hay que **seleccionar un subconjunto de elementos para maximizar un beneficio sin exceder una capacidad limitada**: logística, asignación de recursos, presupuestos, etc. Este informe documenta la resolución del problema clásico de la **Mochila 0/1** (*0/1 Knapsack*) mediante **Programación Dinámica (DP)**, aplicado al escenario de una organización humanitaria que debe cargar un vehículo de capacidad limitada con los recursos más valiosos para una misión.

2. Descripción del problema

Se dispone de N recursos. Cada recurso i tiene un **peso** p_i y un **valor de prioridad** v_i . El vehículo (la mochila) admite un peso total máximo W . Como los recursos no se pueden fraccionar, cada uno se lleva **completo (1)** o **no se lleva (0)**

- de ahí el nombre “0/1”.

Objetivo: elegir el subconjunto de recursos que **maximice el valor total** sin que la suma de sus pesos supere W .

Dataset inicial (capacidad $W = 20$):

Recurso	Peso	Valor
Agua	2	6
Medicamentos	3	8
Alimentos	4	7
Radio	5	10

Generador	9	15
Herramientas	7	12

3. Diseño de la solución

3.1. Estado

Definimos el subproblema:

$dp[i][w]$ = **máximo valor** alcanzable considerando **solo los primeros i objetos** y con una **capacidad disponible w** .

3.2. Recurrencia

Para cada objeto i (peso p_i , valor v_i) y cada capacidad w , tomamos la mejor de dos decisiones:

- **No incluir** el objeto i : $dp[i][w] = dp[i-1][w]$
- **Incluir** el objeto i (solo si $p_i \leq w$): $dp[i-1][w - p_i] + v_i$

$$dp[i][w] = \begin{cases} dp[i-1][w] & \text{si } p_i > w \\ \max(dp[i-1][w], dp[i-1][w - p_i] + v_i) & \text{si } p_i \leq w \end{cases}$$

Casos base: $dp[0][w] = 0$ (sin objetos) y $dp[i][0] = 0$ (sin capacidad).

Respuesta: el valor óptimo queda en $dp[N][W]$.

3.3. Reconstrucción de la solución

La tabla da el *valor* óptimo pero no *qué* objetos se usaron. Se recuperan recorriendo la tabla hacia atrás desde $dp[N][W]$: si $dp[i][w] \neq dp[i-1][w]$, el objeto i **fue usado** → se agrega a la solución y se descuenta su peso ($w \leftarrow w - p_i$). Si son iguales, no se usó. Se repite hasta la fila 0.

3.4. Dos enfoques de DP implementados

- **Bottom-Up** (`mochila_dp.py`): llena la tabla iterativamente desde los casos base. Recorre siempre las $(N+1) \cdot (W+1)$ celdas.

- **Top-Down / Memoization** (`mochila_memoization.py`): recursión que resuelve la recurrencia “natural” y **guarda cada subproblema** la primera vez que lo calcula. Solo calcula los subproblemas realmente alcanzados.

4. Resultados

4.1. Dataset inicial (Partes A, B, C)

Tabla DP generada (filas = objetos considerados; columnas = capacidad `w` de 0 a 20; cada celda = mejor valor con esos objetos y esa capacidad):

Objeto \ w				0	1	2	3	4	5	6	7	8	9	10	11	12	13
14	15	16	17	18	19	20											
(0)	sin obj.			0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0											
(1)	Agua			0	0	6	6	6	6	6	6	6	6	6	6	6	6
6	6	6	6	6	6	6											
(2)	Medicamen			0	0	6	8	8	14	14	14	14	14	14	14	14	14
14	14	14	14	14	14	14											
(3)	Alimentos			0	0	6	8	8	14	14	15	15	21	21	21	21	21
21	21	21	21	21	21	21											
(4)	Radio			0	0	6	8	8	14	14	16	18	21	24	24	25	25
31	31	31	31	31	31	31											
(5)	Generador			0	0	6	8	8	14	14	16	18	21	24	24	25	25
31	31	31	33	36	39	39											
(6)	Herramien			0	0	6	8	8	14	14	16	18	21	24	24	26	26
31	31	33	36	36	39	39											

- **Valor máximo alcanzable: 39** (celda inferior derecha `dp[6][20]`).
- **Objetos seleccionados: Agua + Medicamentos + Radio + Generador.**
- **Peso total: 19** (de 20 disponibles) • **Valor total: 39.**

Nota sobre el enunciado: la Parte B muestra un ejemplo con valor **36** (Agua + Medicamentos + Radio + Herramientas). Ese ejemplo ilustra únicamente el *formato de salida*; **no es el óptimo real** de este dataset. El óptimo verdadero calculado por la DP es **39**, y ambas implementaciones (bottom-up y top-down) coinciden en ese valor.

4.2. Experimento 1 - Variar N (capacidad fija $W = 1000$) • Parte D

N objetos	Capacidad	Tiempo (s)	Tamaño tabla	Valor óptimo
10	1000	0.005723	11 011	368
20	1000	0.004974	21 021	855
50	1000	0.011801	51 051	2731
100	1000	0.023401	101 101	3809
200	1000	0.046110	201 201	5083

Gráfico: `tiempo_vs_n.png`.

4.3. Experimento 2 - Variar W (objetos fijos $N = 100$) • Parte E

N objetos	Capacidad	Tiempo (s)	Tamaño tabla	Valor óptimo
100	250	0.005446	25 351	1701
100	500	0.011539	50 601	2507
100	1000	0.023156	101 101	3701
100	2000	0.048132	202 101	5093
100	4000	0.096596	404 101	5304
100	8000	0.200112	808 101	5304

Gráfico: `tiempo_vs_w.png`.

4.4. Experimento 3 - Bottom-Up vs Top-Down ($W = 1000$) • Desafío opcional

N	Tiempo BU (s)	Tiempo TD (s)	Memoria BU (KB)	Memoria TD (KB)	Mismo óptimo
10	0.002425	0.000296	251.3	88.5	Sí
20	0.004715	0.002560	589.2	225.5	Sí
50	0.011673	0.013637	1423.5	1003.5	Sí
100	0.023462	0.073057	2447.6	2226.5	Sí
200	0.070134	0.165724	4230.6	4045.8	Sí

Gráfico: `bottomup_vs_topdown.png` .

Nota metodológica: el tiempo se midió **sin** `tracemalloc` (que ralentiza mucho) y la memoria en una pasada aparte **con** `tracemalloc` , para no contaminar los tiempos.

5. Análisis de complejidad (Parte E)

Complejidad teórica: la tabla tiene $(N+1) \cdot (W+1)$ celdas y cada celda se calcula en tiempo constante $O(1) \rightarrow$ **tiempo** $O(N \cdot W)$ y **memoria** $O(N \cdot W)$. Es *pseudopolinómica*: depende del **valor** de W , no solo de la cantidad de datos.

¿Cómo crece el tiempo al aumentar N?

De forma **aproximadamente lineal**. Con W fijo, pasar de $N=50$ a $N=100$ a $N=200$ llevó el tiempo de $0.012\text{ s} \rightarrow 0.023\text{ s} \rightarrow 0.046\text{ s}$: **cada vez que N se duplica, el tiempo se duplica**, como predice $O(N \cdot W)$.

¿Cómo afecta la capacidad W al tiempo?

También de forma **lineal**, y es el factor más notorio. En el Experimento 2, duplicar W ($250 \rightarrow 500 \rightarrow 1000 \rightarrow \dots \rightarrow 8000$) duplicó el tiempo en cada paso ($0.005 \rightarrow 0.012 \rightarrow 0.023 \rightarrow 0.048 \rightarrow 0.097 \rightarrow 0.200\text{ s}$). El gráfico `tiempo_vs_w.png` es prácticamente una **recta**. Observación interesante: a partir de $W = 4000$ el **valor óptimo se estabiliza en 5304**, porque la mochila ya es lo bastante grande para llevar *todos* los objetos; aun así el

tiempo sigue creciendo, confirmando que el costo depende del **tamaño de la tabla**, no de cuán restrictiva sea la capacidad.

¿Coinciden los resultados con la complejidad teórica $O(N \cdot W)$?

Sí. El tiempo escala linealmente tanto con N como con W por separado, que es exactamente el comportamiento de un producto $N \cdot W$. Las pequeñas irregularidades (p. ej. $N=10$ vs $N=20$) se deben a ruido del sistema operativo y a tiempos muy pequeños, no a un cambio de tendencia.

¿Qué ventajas tiene DP frente a la búsqueda exhaustiva?

La búsqueda exhaustiva prueba **todos los subconjuntos**: $O(2^N)$. Para $N=200$ eso son $2^{200} \approx 10^{60}$ combinaciones - **imposible** en la práctica. La DP lo resuelve en $N \cdot W = 200 \cdot 1000 = 200\,000$ operaciones (fracciones de segundo) porque **reutiliza los subproblemas superpuestos** en lugar de recalcularlos. Esa es la esencia de la Programación Dinámica.

6. Desafío opcional - Bottom-Up vs Top-Down

- **Tiempo:** con **pocos objetos** ($N=10, 20$) Top-Down es más rápido, porque solo calcula los subproblemas necesarios (en el dataset inicial, 44 de 147 celdas). Con **muchos objetos** ($N=100, 200$) Bottom-Up gana, porque los bucles iterativos evitan el **overhead de las llamadas recursivas** de Python.
- **Memoria:** Top-Down consumió algo **menos** de memoria pico, ya que no siempre necesita materializar toda la tabla; a cambio usa la **pila de recursión** (que crece con N y puede requerir subir el límite de recursión).
- **Facilidad de implementación:** Top-Down suele ser más **directo de escribir** (se traduce casi literalmente la recurrencia); Bottom-Up obliga a pensar el **orden de llenado**, pero es más **predecible y robusto** (sin riesgo de desbordar la pila).
- **Corrección:** ambos enfoques dan **exactamente el mismo valor óptimo** en todos los casos, como se ve en la columna “Mismo óptimo = Sí”.

Conclusión del desafío: son dos caras de la misma técnica. Top-Down es cómodo y eficiente cuando pocos subproblemas se necesitan; Bottom-Up es preferible para entradas grandes por su menor overhead y su seguridad frente a la recursión profunda.

7. Conclusiones (lecciones aprendidas)

1. **Identificar subproblemas superpuestos** es la clave para aplicar DP: aquí, la mejor carga con menos objetos y menos capacidad se reutiliza constantemente.
2. La **recurrencia** “**meter o no meter**” más una **tabla** convierten un problema exponencial (2^N) en uno **pseudopolinómico** ($O(N \cdot W)$), perfectamente ejecutable.
3. La **tabla DP no solo da el valor óptimo**: recorriéndola hacia atrás recuperamos *qué* objetos componen la solución.
4. Los **experimentos confirman la teoría**: el tiempo crece linealmente con N y con W , validando $O(N \cdot W)$ de forma empírica.
5. **Bottom-Up y Top-Down** son equivalentes en resultado; la elección depende del tamaño del problema y de las restricciones de memoria/recursión.